

LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)

Approved by AICTE | Affiliated to Osmania University

Himayath Sagar, Hyderabad – 500091

Department of CSE



Full Stack Web Development Lab Manual

IV. B.E. – VIIth Semester

LR- 23

A.Y: 2026-2027

SUBJECT CODE - U23CS7L2

LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)

Approved by AICTE | Affiliated to Osmania University

Himayath Sagar, Hyderabad – 500091



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC AUTONOMOUS)

Approved by AICTE | Affiliated to Osmania University | Estd. 2003

Department of CSE

Certificate

This is to certify that **Ms. Mubeen Khan** Assistant Professor, **Department of CSE** has completed all the requirements of the Lab Manual as per the contents given of **“Full Stack Web Development Lab” (U23CS7L1)** during Academic Year **2026 - 2027**, B.E (CSE) VII -Semester.

Course Coordinator

Head of the Department

INDEX

S.No	Description
1.	Institute Vision and Mission
2.	Department Vision and Mission, PEOs, POs and PSOs
3.	Lab Syllabus
4.	COs of the lab
5.	CO-PO mappings of the lab
6.	Lab experiments aim, procedures, diagrams, tables etc
7.	Viva questions at the end of each experiment

	List of Experiments
1.	Building a Personal Portfolio Page with HTML using necessary tags, consists of image, form, table for display skills and ratings.
2.	Styling the Portfolio with CSS Box Model, Flexbox & Project Gallery using Grid. It should be responsive use media query tags.
3.	Java script basics - explore operators, control flow, arrays, functions, and string manipulation.
4.	Bring the portfolio to life: dynamically add/remove project cards, change themes, and respond to user events.
5.	Validate the Contact form from Exp 01 using regex patterns; implement try-catch exception handling.
6.	Fetch API - Get live data (e.g., GitHub user stats, etc) and render it on the portfolio, introducing async/await.
7.	React.js Re-build the portfolio as a React app with functional components, props, state (useState), and useEffect.
8.	Node.js Install Node.js, understand the event loop, create a basic HTTP server, and use core modules.
9.	Express.js Build a RESTful API that serves portfolio data (projects, messages) and handles Contact-form submissions.
10.	MongoDB & Mongoose Connect the Express API to MongoDB Atlas; perform full CRUD on Projects and Contact collections.
11.	Full-Stack Integration - Integrate React (Exp 07) + Express (Exp 09) + MongoDB (Exp 10) into a single deployable MERN application.
12.	Deployment - Deploy the full MERN portfolio application to the cloud so it is publicly accessible.



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY **(UGC Autonomous)**

Approved by AICTE | Affiliated to Osmania University | Estd.2003.

Accredited with 'A' grade by NAAC | Accredited by NBA

Vision of the Institute:

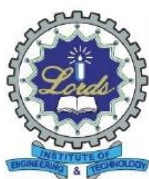
Lords Institute of Engineering and Technology strives for excellence in professional education through quality, innovation and teamwork and aims to emerge as a premier institute in the state and across the nation.

Mission of the Institute:

- To impart quality professional education that meets the needs of present and emerging technological world.
- To strive for student achievement and success, preparing them for life, career and leadership.
- To provide a scholarly and vibrant learning environment that enables faculty, staff and students to achieve personal and professional growth.
- To contribute to advancement of knowledge, in both fundamental and applied areas of engineering and technology.
- To forge mutually beneficial relationships with government organizations, industries, society and the alumni.

Principal

Lords Institute of Engineering & Tech.
(An Autonomous Institution)
Sy. No. 32, Himayath Sagar, Hyderabad-500091.T.S.



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)

Approved by AICTE | Affiliated to Osmania University | Estd.2003.

Accredited with 'A' grade by NAAC | Accredited by NBA

Department of Computer Science and Engineering

Vision of the Department:

To emerge as a center of excellence for quality Computer Science and Engineering education with innovation, leadership and values.

Mission of the Department:

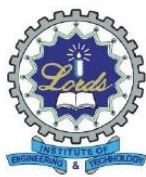
DM1: Provide fundamental and practical training through learner – centric Teaching-Learning Process and state-of-the-art infrastructure.

DM2: Develop design, research, and entrepreneurial skills for successful career

DM3: Promote training and activities through Industry-Academia interactions.

Note: DM: Department Mission

HoD



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)

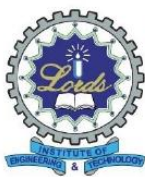
Approved by AICTE | Affiliated to Osmania University | Estd.2003.

Accredited with 'A' grade by NAAC | Accredited by NBA

Department of Computer Science and Engineering

B.E. Computer Science and Engineering Program Educational Objectives (PEOs):

PEO1	Exhibit strong foundations in Basic Sciences, Computer Science and allied engineering
PEO2	Identify, formulate, analyze and create professional solutions, novel products using appropriate tools and techniques, design skills.
PEO3	Pursue successful career with continuous learning, with emphasis on competency oriented towards industry
PEO4	Practice ethics, managerial and leadership skills to work cohesively within a group



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)

Approved by AICTE | Affiliated to Osmania University | Estd.2003.

Accredited with 'A' grade by NAAC | Accredited by NBA

Department of Computer Science and Engineering

B.E.Computer Science and Engineering Program Outcomes (POs):

Engineering Graduates will be able to:

S. No.	Program Outcomes (POs)
1.	PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
2.	PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
3.	PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
4.	PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
5.	PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
6.	PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
7.	PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
8.	PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
9.	PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
10.	PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
11.	PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Knowledge and Attitude Profile(WK)

WK's	Description
WK1	A systematic, theory-based understanding of the natural sciences applicable to the discipline and awareness of relevant social sciences.
WK2	Conceptually-based mathematics, numerical analysis, data analysis, statistics and formal aspects of computer and information science to support detailed analysis and modelling applicable to the discipline.
WK3	A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline.
WK4	Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.
WK5	Knowledge, including efficient resource use, environmental impacts, whole-life cost, re-use of resources, net zero carbon, and similar concepts, that supports engineering design and operations in a practice area.
WK6	Knowledge of engineering practice(technology) in the practice areas in the engineering discipline.
WK7	Knowledge of the role of engineering in society and identified issues in engineering practice in the discipline, such as the professional responsibility of an engineer to public safety and sustainable development.
WK8	Engagement with selected knowledge in the current research literature of the discipline, awareness of the power of critical thinking and creative approaches to evaluate emerging issues.
WK9	Ethics, inclusive behavior and conduct. Knowledge of professional ethics, responsibilities, and norms of engineering practice. Awareness of the need for diversity by reason of ethnicity, gender, age, physical ability etc. with mutual understanding and respect, and of inclusive attitudes.

Program Outcome	WK1	WK2	WK3	WK4	WK5	WK6	WK7	WK8	WK9
PO1 Engineering Knowledge									
PO2 Problem Analysis									
PO3 Design/Development of Solutions									
PO4 Conduct Investigations Of Complex Problems									
PO5 Engineering Tool Usage									
PO6 The Engineer & The World									
PO7 Ethics									
PO8 Individual & collaborative Team work									
PO9 Communication									
PO10 Project Management And Finance									
PO11 Life-Long Learning									



LORDS INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC Autonomous)

Approved by AICTE | Affiliated to Osmania University | Estd.2003.

Accredited with 'A' grade by NAAC | Accredited by NBA

Department of Computer Science and Engineering

B.E. Computer Science and Engineering Program Specific Outcomes (PSO's):

PSO1	Professional Skills: Implement computer programs in the areas related to algorithms, system software, multimedia, web design, big data analytics and networking for efficient analysis and design of computer-based systems of varying complexity
PSO2	Problem-Solving Skills: Apply standard practices and strategies in software service management using open-ended programming environment with agility to deliver a quality service for business success

Course Code	Course Title				Core/Elective	
U23CS7L1	FULL STACK WEB DEVELOPMENT LAB				Core Course	
Prerequisites	Contact Hour per Week			CIE	SEE	Credit
	L	T	P			
HTML/CSS Basic	-	-	3	40	60	1.5

Course Objectives:

1. Understand HTML, CSS, and JavaScript for web development
2. Develop responsive and interactive web applications.
3. Build front-end applications using React.js.
4. Create back-end services using Node.js, Express.js, and MongoDB.
5. Integrate and deploy full-stack MERN applications.

Course Outcomes:

At the end of the course, student would be able to

1. Design responsive web pages using Semantic HTML and CSS.
2. Develop interactive web applications using JavaScript.
3. Build dynamic user interfaces using React.js.
4. Develop RESTful APIs and database-driven applications using the MERN stack.
5. Integrate and deploy full-stack web applications on cloud platforms.

LAB EXPERIMENT**1. HTML**

Building a Personal Portfolio Page with HTML using necessary tags, consists of image, form, table for display skills and ratings.

2. CSS

Styling the Portfolio with CSS – Box Model, Flexbox & Project Gallery using Grid. It should be responsive use media query tags.

3. **Java script** basics – explore operators, control flow, arrays, functions, and string manipulation .
4. Bring the portfolio to life: dynamically add/remove project cards, change themes, and respond to user events.
5. Validate the Contact form from Exp 01 using regex patterns; implement try-catch exception handling.
6. Fetch API - Get live data (e.g., GitHub user stats, etc) and render it on the portfolio, introducing async/await.
7. **React.js** - Re-build the portfolio as a React app with functional components, props, state (useState), and useEffect.
8. **Node.js** - Install Node.js, understand the event loop, create a basic HTTP server, and use core modules.
9. **Express.js** - Build a RESTful API that serves portfolio data (projects, messages) and handles Contact-form submissions.
10. **MongoDB & Mongoose** - Connect the Express API to MongoDB Atlas; perform full CRUD on Projects and Contact collections.
11. **Full-Stack Integration** - Integrate React (Exp 07) + Express (Exp 09) + MongoDB (Exp 10) into a single deployable MERN application.
12. **Deployment** - Deploy the full MERN portfolio application to the cloud so it is publicly accessible.

Suggested Readings:

1. Vasan Subramanian, "Pro MERN Stack: Full Stack Web App Development with Mongo, Express, React, and Node", 2nd Edition, Apress.
2. Thomas A Powel "The Complete Reference HTML & CSS", 5th Edition, McGraw Hill.
3. Ketan Agnihotri, Pranali Dahale, " React Development using Typescript: Modern web app development using advanced React techniques ", English Edition, 2024.
4. Nabendu Biswas, "Ultimate Full-Stack Web Development with MERN: Design, Build, Test and Deploy Production-Grade Applications with MongoDB, Express, React and NodeJS", 1st Edition, 2023.

Online Resources:

1. <https://react.dev/>
2. <https://nodejs.org/learn/>
3. <https://expressjs.com/en/>
4. <https://www.mongodb.com/docs/>
5. <https://www.w3schools.com/>

EXPERIMENT 1

Personal Portfolio Page using HTML

Aim

To design and develop a personal portfolio web page using HTML, incorporating semantic tags, an image, a form, and a table to display skills and ratings.

Description

This experiment introduces the foundational structure of a web page using HTML5. Students create a single-page portfolio consisting of standard sections — header/navigation, about, skills, projects, and contact — using semantic elements such as header, nav, section, article, and footer. The page must include an image element for a profile photo, an HTML form for a contact section (name, email, message fields), and an HTML table to display a list of skills alongside proficiency ratings.

Algorithm

1. Create a new HTML5 boilerplate file (index.html) with the DOCTYPE declaration, html, head, and body elements.
2. Add metadata in the head section: title, character-set meta tag, and viewport meta tag.
3. Build a header containing the student's name and a nav element with anchor links to page sections (About, Skills, Projects, Contact).
4. Create an About section with a profile image tag and a short bio paragraph.
5. Create a Skills section containing a table with columns for Skill Name and Rating.
6. Create a Projects section using article elements as placeholders for each project (to be styled as a gallery in Experiment 2).
7. Create a Contact section with a form containing a text input for name, an email input, a textarea for the message, and a submit button.
8. Add a footer with copyright text.
9. Validate the HTML using the W3C Markup Validator.
10. Save and open the file in a browser to verify the structure and content render correctly.

Viva Questions & Answers

Viva Voce Questions & Answers

Q1. What is the difference between an HTML element and an HTML tag?

Viva Voce Questions & Answers

Ans: A tag is the markup syntax (such as <p>) used to open or close an element, while an element is the complete unit consisting of the opening tag, its content, and the closing tag.

Q2. Why are semantic tags like header, nav, and footer preferred over generic div tags?

Ans: Semantic tags convey meaning about their content to browsers, search engines, and assistive technologies, improving accessibility and SEO, whereas a div carries no inherent meaning.

Q3. What is the purpose of the action and method attributes of a form tag?

Ans: The action attribute specifies the URL where form data is submitted, and the method attribute (GET or POST) defines how the data is sent to the server.

Q4. What is the difference between the src and alt attributes of an img tag?

Ans: src specifies the path or URL of the image to display, while alt provides alternative text that is shown if the image fails to load and is read by screen readers.

Q5. Why is a table considered appropriate for displaying skills and ratings, but discouraged for full page layout?

Ans: Tables are semantically correct for tabular data such as skill-rating pairs, but using them for page layout mixes structure with presentation and breaks responsiveness, which CSS layout tools handle far better.

Q6. What does the DOCTYPE declaration do at the top of an HTML file?

Ans: It instructs the browser to render the page in HTML5 standards mode, ensuring consistent rendering behaviour across different browsers.

EXPERIMENT 2

Styling the Portfolio with CSS (Box Model, Flexbox & Grid)

Aim

To style the personal portfolio page using CSS, applying the Box Model, Flexbox for layout, and CSS Grid for a responsive project gallery, using media queries for responsiveness.

Description

This experiment builds on Experiment 1 by applying visual design principles. Students learn the CSS Box Model (content, padding, border, margin) to control spacing, use Flexbox to align the navigation bar and form elements, and use CSS Grid to build a multi-column project gallery that reflows responsively across screen sizes with the help of media queries.

Algorithm

1. Create a styles.css file and link it to index.html using a link tag.
2. Apply a universal box-sizing: border-box rule to normalise the box model across all elements.
3. Style the body with a base font-family, colour scheme, and line-height.
4. Style the nav using display: flex along with justify-content and align-items to align navigation links horizontally.
5. Style the Projects section container using display: grid with grid-template-columns set to repeat(auto-fit, minmax(...)) to create a responsive gallery.
6. Add padding, margin, and border styling to each project card, and use box-shadow for visual depth.
7. Style the contact form using Flexbox with flex-direction: column to stack labels and inputs cleanly.
8. Add a media query (for example, max-width: 768px) to collapse the navigation and switch the gallery to a single column on smaller screens.
9. Test the page at multiple viewport widths using the browser's responsive design mode.
10. Validate the CSS using the W3C CSS Validator and refine spacing and alignment.

Viva Questions & Answers

Viva Voce Questions & Answers

Q1. What are the four components of the CSS Box Model, from innermost to outermost?

Ans: Content, Padding, Border, and Margin.

Q2. What is the difference between box-sizing: content-box and box-sizing: border-box?

Ans: content-box (the default) adds padding and border outside the specified width, increasing the element's total rendered size; border-box includes padding and border within the specified width, keeping the total size fixed.

Q3. What is the main difference between Flexbox and CSS Grid?

Ans: Flexbox is a one-dimensional layout system best suited for arranging items in a single row or column, while Grid is a two-dimensional system that controls rows and columns simultaneously, making it better suited to gallery-style layouts.

Q4. What does justify-content control in a Flexbox container, and how does it differ from align-items?

Ans: justify-content aligns flex items along the main axis (horizontal by default), while align-items aligns items along the cross axis (vertical by default).

Q5. What is a media query, and why is it essential for responsive design?

Ans: A media query applies CSS rules conditionally based on device characteristics such as viewport width, allowing a single stylesheet to adapt the layout for different screen sizes.

Q6. What does grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)) achieve?

Ans: It creates as many columns as will fit the container, each at least 200px wide, and distributes any remaining space evenly, producing a responsive grid without needing separate media queries for column count.

EXPERIMENT 3

JavaScript Basics — Operators, Control Flow, Arrays, Functions, Strings

Aim

To explore core JavaScript fundamentals — operators, control flow, arrays, functions, and string manipulation — through standalone exercises.

Description

This experiment introduces JavaScript as a programming language independent of the DOM. Students practise arithmetic, comparison, and logical operators; conditional statements and loops; array creation and common iteration methods; function declarations, expressions, and arrow functions; and frequently used string manipulation methods.

Algorithm

1. Create a script.js file and link it to a test HTML page using a script tag with the defer attribute, or run it directly using Node.js from the terminal.
2. Declare variables using let and const, and demonstrate arithmetic, comparison, and logical operators using console.log statements.
3. Write an if-else statement and a switch statement to demonstrate branching logic, such as grading a numeric score.
4. Write a for loop and a while loop that iterate over a range of numbers or an array.
5. Declare an array of sample data (for example, project names or skill ratings) and demonstrate the push, pop, map, filter, and forEach array methods.
6. Write a function declaration and an equivalent arrow function that perform the same task, such as calculating an average rating.
7. Demonstrate string manipulation: concatenation, template literals, split, trim, toUpperCase, and slice.
8. Combine the above into a short script that processes an array of skill objects and logs a formatted summary string for each.
9. Test each snippet in the browser console or the Node.js REPL and verify the expected output.

Viva Questions & Answers

Viva Voce Questions & Answers

Q1. What is the difference between let, const, and var?

Ans: var is function-scoped and can be redeclared and updated; let is block-scoped and can be updated but not redeclared in the same scope; const is block-scoped and cannot be reassigned after initialisation.

Q2. What is the difference between == and === in JavaScript?

Ans: == performs type coercion before comparing values, while === (strict equality) compares both value and type without coercion.

Q3. What is the difference between map() and forEach() on an array?

Ans: map() returns a new array containing the results of applying a function to each element, while forEach() simply executes a function for each element and returns undefined.

Q4. What is a template literal, and what advantage does it offer over string concatenation?

Ans: A template literal uses backticks and allows embedded expressions, making string interpolation cleaner and more readable than repeated use of the + operator.

Q5. What is the difference between a function declaration and an arrow function with respect to 'this'?

Ans: A regular function declaration has its own 'this' binding determined by how it is called, while an arrow function has no 'this' of its own and instead inherits 'this' from its enclosing lexical scope.

Q6. What does the filter() array method do?

Ans: It returns a new array containing only the elements that satisfy a given test function, without modifying the original array.

EXPERIMENT 4

DOM Manipulation & Events

Aim

To bring the static portfolio to life using JavaScript DOM manipulation — dynamically adding and removing project cards, switching themes, and responding to user events.

Description

This experiment connects JavaScript to the HTML structure via the Document Object Model. Students select and manipulate DOM elements, create and remove project cards dynamically from a JavaScript array of data, implement a light/dark theme toggle by adding and removing CSS classes, and attach event listeners to respond to clicks and user input.

Algorithm

1. Select key DOM elements such as the project gallery container and the theme toggle button using query selector methods.
2. Create a JavaScript array of project objects containing a title, description, and image URL.
3. Write a renderProjects function that loops through the array, dynamically creates elements for each project, sets their content, and appends them to the gallery container.
4. Add a Remove button to each generated card and attach a click event listener that removes the corresponding card from the DOM.
5. Add an Add Project control that pushes a new object into the array and re-renders the gallery.
6. Implement a theme toggle button with an event listener that toggles a dark-theme class on the body element.
7. Store the selected theme so the choice persists across interactions during the session.
8. Attach event listeners for additional user events, such as a hover effect on cards or an input event on a search box to filter visible cards.
9. Test all interactions in the browser and verify the DOM updates correctly without page reloads.

Viva Questions & Answers

Viva Voce Questions & Answers

Q1. What is the DOM, and how does it relate to an HTML document?

Viva Voce Questions & Answers

Ans: The Document Object Model is a tree-like, in-memory representation of the HTML document created by the browser, where each HTML tag becomes a node that JavaScript can access and manipulate.

Q2. What is the difference between `innerHTML` and `createElement()` when adding content to a page?

Ans: `innerHTML` parses a string as HTML and injects it directly, which is quick but risks security issues with untrusted input, while `createElement()` builds DOM nodes programmatically, giving safer and finer control over each element.

Q3. What is event delegation, and why is it useful for dynamically created elements?

Ans: Event delegation attaches a single listener to a parent element and inspects the event target to determine which child triggered it, allowing events on elements added to the DOM later to be handled without attaching new listeners each time.

Q4. What does `classList.toggle()` do, and how is it used for a theme switcher?

Ans: It adds a class if it is not present and removes it if it is, so a single call can switch between light and dark themes each time it is invoked.

Q5. What is the difference between `addEventListener()` and an inline `onclick` attribute?

Ans: `addEventListener()` separates JavaScript from HTML, supports multiple listeners on the same element and event, and offers finer options, whereas an inline `onclick` attribute mixes markup with logic and supports only one handler per event.

Q6. What is the difference between `event.target` and `event.currentTarget`?

Ans: `event.target` refers to the actual element that triggered the event, while `event.currentTarget` refers to the element the listener is attached to; these can differ during event bubbling or delegation.

EXPERIMENT 5

Contact Form Validation using Regex & Exception Handling

Aim

To validate the Contact form built in Experiment 1 using JavaScript regular expressions and implement try-catch based exception handling.

Description

This experiment focuses on client-side form validation. Students write regex patterns to validate the name, email, and message fields, display inline error messages, prevent form submission when validation fails, and use try-catch blocks with custom error throwing to handle invalid or unexpected input gracefully.

Algorithm

1. Select the contact form and its input fields using DOM methods.
2. Attach a submit event listener to the form and prevent the default submission behaviour while validation runs.
3. Define regex patterns: a name pattern allowing letters and spaces, an email pattern matching a standard email format, and a minimum-length check for the message field.
4. Write a validation function that returns true or false based on testing a field's value against the relevant pattern.
5. For each field, display an inline error message and apply an error style if validation fails, or clear any existing error if it passes.
6. Wrap the overall validation logic in a try block, and throw a custom error for specific invalid conditions such as an empty required field.
7. Add a catch block that catches thrown errors, logs them, and displays a general form-level error message to the user.
8. Add a finally block to reset the submitting state or UI regardless of whether validation succeeded or failed.
9. On successful validation, show a confirmation message (later replaced by an actual submission in Experiments 6 and 9).
10. Test with valid inputs, invalid inputs, and edge cases such as empty fields or a malformed email to confirm correct handling.

Viva Questions & Answers

Viva Voce Questions & Answers

Q1. What is a regular expression, and why is it useful for form validation?

Ans: A regular expression is a pattern used to match character combinations in strings; it lets developers concisely check whether input such as an email or name conforms to an expected format.

Q2. What is the difference between test() and match() when using regex in JavaScript?

Ans: The regex test() method returns a boolean indicating whether a match exists, while the string match() method returns an array of match details or null, making test() more efficient for simple validity checks.

Q3. What is the purpose of a try-catch block?

Ans: A try block contains code that might throw an error; if an error occurs, control passes to the catch block, allowing the program to handle the failure gracefully instead of crashing.

Q4. What does the finally block do, and when does it execute?

Ans: The finally block contains code that always executes after the try and catch blocks, regardless of whether an error was thrown or caught, and is commonly used for cleanup actions.

Q5. How do you create and throw a custom error in JavaScript?

Ans: By using the throw keyword with a new Error object carrying a custom message, which can then be caught and inspected in a catch block through its message property.

Q6. Why is client-side validation not sufficient on its own, and what should accompany it?

Ans: Client-side validation can be bypassed by disabling JavaScript or sending requests directly to the server, so it must always be accompanied by server-side validation to ensure data integrity and security.

EXPERIMENT 6

Fetch API & Async/Await

Aim

To fetch live data from a public API, such as GitHub user statistics, and render it on the portfolio using the Fetch API and async/await.

Description

This experiment introduces asynchronous JavaScript. Students use the Fetch API to make an HTTP GET request to a public REST endpoint, handle the returned Promise using async/await, parse the JSON response, and dynamically render the data onto the portfolio page, along with error handling for failed or slow network requests.

Algorithm

1. Identify a public API endpoint, such as the GitHub Users API, and inspect the structure of its JSON response.
2. Create an async function that accepts a username as a parameter.
3. Inside the function, use fetch with await to send the GET request and store the response.
4. Check whether the response is successful; if not, throw an error to be handled by the caller.
5. Parse the response body as JSON using await, again handled as an asynchronous operation.
6. Use the parsed data to update DOM elements, such as setting text content for a followers count or the source of an avatar image.
7. Wrap the fetch and parsing logic in a try-catch block to handle network failures or invalid usernames, displaying a fallback message on error.
8. Call the async function when the page loads or when a button is clicked, and show a loading state while the request is pending.
9. Test with valid and invalid usernames, and simulate a network failure to verify the error handling works correctly.

Viva Questions & Answers

Viva Voce Questions & Answers

Q1. What is a Promise in JavaScript?

Viva Voce Questions & Answers

Ans: A Promise is an object representing the eventual completion or failure of an asynchronous operation, existing in one of three states: pending, fulfilled, or rejected.

Q2. How does async/await simplify working with Promises compared to .then() chains?

Ans: async/await lets asynchronous code be written in a synchronous-looking style, improving readability and making error handling straightforward using try-catch, rather than chaining multiple .then() and .catch() calls.

Q3. Why must fetch() responses be explicitly checked for a successful status?

Ans: fetch() only rejects its Promise on a network failure, not for HTTP error statuses such as 404 or 500, so the response status must be checked manually to detect and handle server-side errors.

Q4. Why is parsing the response body as JSON also an asynchronous operation?

Ans: Because reading the response stream itself takes time and is handled asynchronously, so it also returns a Promise that must be awaited or resolved before the data can be used.

Q5. What is the difference between synchronous and asynchronous code execution in JavaScript?

Ans: Synchronous code executes line by line, blocking further execution until the current operation completes, while asynchronous code allows long-running operations, such as network requests, to run in the background without blocking the main thread.

Q6. What happens if an awaited expression inside an async function throws an error and there is no try-catch?

Ans: The async function's returned Promise becomes rejected with that error, and if the rejection is not handled elsewhere, it results in an unhandled promise rejection.

EXPERIMENT 7

Rebuilding the Portfolio with React.js

Aim

To rebuild the personal portfolio as a React application using functional components, props, state (useState), and effects (useEffect).

Description

This experiment migrates the vanilla HTML, CSS, and JavaScript portfolio into a component-based React architecture. Students scaffold a React app, break the portfolio into reusable functional components, pass data between components using props, manage local component state with useState, and use useEffect to replicate the Fetch API call from Experiment 6 as a side effect.

Algorithm

1. Scaffold a new React project using a build tool such as Vite or Create React App.
2. Plan and create a component hierarchy: an App root component with child components such as Header, About, SkillsTable, ProjectGallery, ProjectCard, and ContactForm.
3. Convert the static HTML sections into JSX within their respective functional components.
4. Pass the projects array and skills array as props from App down to the relevant components, rendering lists with a unique key on each item.
5. Use the useState hook inside ContactForm to manage controlled input values and validation error state, reusing the logic from Experiment 5.
6. Use the useState hook to manage the current theme, and pass the theme value and a toggle function down as props.
7. Use the useEffect hook to call the GitHub fetch logic from Experiment 6 once the component mounts, storing the result in state.
8. Style the components using the CSS from Experiment 2, adapted as component-scoped styles.
9. Run the development server and verify each component renders and updates correctly as props and state change.

Viva Questions & Answers

Viva Voce Questions & Answers

Q1. What is the difference between a functional component and props in React?

Viva Voce Questions & Answers

Ans: A functional component is a JavaScript function that returns JSX describing the UI, while props are read-only inputs passed into that function from a parent component to configure its output.

Q2. Why does React require a unique key when rendering lists?

Ans: The key helps React's reconciliation process identify which items have changed, been added, or removed between renders, allowing efficient DOM updates instead of re-rendering the entire list.

Q3. What does the useState hook return, and how is it typically used?

Ans: useState returns an array containing the current state value and a setter function, commonly destructured together, where calling the setter triggers a re-render with the updated value.

Q4. What is the purpose of the dependency array in useEffect?

Ans: The dependency array controls when the effect re-runs: an empty array runs the effect only once after the initial render, a populated array re-runs it whenever a listed value changes, and omitting it runs the effect after every render.

Q5. What is the difference between state and props in React?

Ans: Props are passed into a component from its parent and are immutable within that component, while state is data managed internally by a component and can change over time, triggering re-renders.

Q6. Why is React described as using a virtual DOM?

Ans: React maintains an in-memory representation of the UI; when state or props change, it computes the difference between the previous and new representation and applies only the minimal necessary updates to the real DOM, improving performance.

EXPERIMENT 8

Node.js Fundamentals — Event Loop & HTTP Server

Aim

To install Node.js, understand its event loop, create a basic HTTP server, and use core Node modules.

Description

This experiment introduces server-side JavaScript using Node.js. Students install Node.js and verify the installation, understand the single-threaded, non-blocking event loop model, create a basic HTTP server using the built-in http module, and explore other core modules such as the file system and path modules.

Algorithm

1. Download and install the LTS version of Node.js, and verify the installation by checking the Node and npm versions from the terminal.
2. Create a project folder and initialise it to generate a package.json file.
3. Create a server file and require the built-in http module.
4. Use the http module's server-creation function with a callback that sets a response header and sends a response body.
5. Call the server's listen method to start it on a specified port, and log a confirmation message.
6. Run the server and verify the response by visiting the local address in a browser or using a command-line HTTP client.
7. Extend the server to respond differently based on the request URL, implementing basic routing for a few plain-text paths.
8. Demonstrate the event loop by logging statements before and after a delayed callback and a synchronous operation, and observe the execution order.
9. Use the core file system module to read a text file asynchronously and log its content, illustrating Node's non-blocking I/O.
10. Use the core path module to safely construct file paths relative to the current directory.

Viva Questions & Answers

Viva Voce Questions & Answers

Q1. What is Node.js, and how does it allow JavaScript to run outside the browser?

Viva Voce Questions & Answers

Ans: Node.js is a JavaScript runtime built on Google Chrome's V8 engine that executes JavaScript on the server side, providing APIs for tasks browsers do not support, such as file system access and networking.

Q2. What is the event loop, and why is it central to Node.js's concurrency model?

Ans: The event loop allows Node.js, despite being single-threaded, to perform non-blocking I/O by offloading operations such as file reads or network calls to the system and executing their callbacks once complete, without blocking the main thread.

Q3. What is the difference between require() and installing a package with npm?

Ans: require() is the function used to import a module into a file, whereas npm is the tool used to download and manage third-party packages listed in package.json before they can be required.

Q4. What is the purpose of the package.json file?

Ans: It stores metadata about a Node.js project, including its name, version, dependencies, and scripts, and is used by npm to manage installed packages and run project tasks.

Q5. What is the difference between the asynchronous and synchronous versions of a file-read function in Node?

Ans: The asynchronous version is non-blocking and takes a callback that runs once the read completes, while the synchronous version blocks the event loop until the file is fully read.

Q6. What does setting the response headers do in an HTTP server callback?

Ans: It sets the HTTP status code and headers, such as the content type, before the response body is sent, telling the client how to interpret the response.

EXPERIMENT 9

Building a RESTful API with Express.js

Aim

To build a RESTful API using Express.js that serves portfolio data (projects, messages) and handles Contact-form submissions.

Description

This experiment introduces the Express.js framework to simplify server and routing logic compared to raw Node.js. Students set up an Express application, define RESTful routes for retrieving project data and submitting contact messages, use middleware to handle request bodies and cross-origin requests, and structure routes following REST conventions.

Algorithm

1. Initialise a new project and install Express and a CORS-handling package.
2. Create a server file, require Express, and initialise the application.
3. Add middleware to parse JSON request bodies and to allow cross-origin requests from the React frontend.
4. Create an in-memory or static data store for projects and an array to hold submitted contact messages.
5. Create a GET route that returns the list of projects as JSON.
6. Create a POST route that reads the request body, validates the required fields, stores the new message, and responds with a success status.
7. Create a GET route for an admin view that retrieves the submitted messages.
8. Add basic error-handling middleware to catch and respond to unexpected errors with an appropriate status code.
9. Start the server and test each route using a REST client such as Postman or Thunder Client.
10. Update the React ContactForm from Experiment 7 to submit to the new POST route, replacing the client-only validation flow.

Viva Questions & Answers

Viva Voce Questions & Answers

Q1. What is Express.js, and what problem does it solve compared to using the raw http module?

Viva Voce Questions & Answers

Ans: Express.js is a minimal web framework built on top of Node's http module that simplifies routing, middleware handling, and request and response processing, removing the need to manually parse URLs and manage routing logic.

Q2. What is middleware in Express, and how does the next function relate to it?

Ans: Middleware is a function with access to the request, response, and a next callback that executes during the request-response cycle; calling next passes control to the subsequent middleware or route handler, while not calling it halts the request there.

Q3. What is the difference between a GET route and a POST route in Express?

Ans: A GET route handler is typically used to retrieve data, while a POST route handler is typically used to submit or create data, with the submitted payload available on the request body.

Q4. Why is JSON body-parsing middleware necessary when handling POST requests with a JSON payload?

Ans: Without it, Express does not automatically parse the raw request body; the middleware parses incoming JSON payloads and makes the submitted data available to route handlers.

Q5. What does CORS mean, and why is CORS middleware needed when the frontend and backend run on different ports?

Ans: CORS stands for Cross-Origin Resource Sharing, a browser security mechanism that blocks requests between different origins by default; the middleware adds the necessary response headers to explicitly allow the frontend, running on a different origin, to call the API.

Q6. What HTTP status code should a successful POST request that creates a new resource typically return, and why?

Ans: 201 Created, because it specifically indicates that the request succeeded and a new resource was created as a result, distinguishing it from the generic 200 OK response.

EXPERIMENT 10

MongoDB & Mongoose — CRUD Operations

Aim

To connect the Express API to MongoDB Atlas using Mongoose and perform full CRUD operations on the Projects and Contact collections.

Description

This experiment replaces the in-memory data store from Experiment 9 with persistent storage in MongoDB. Students set up a free MongoDB Atlas cluster, connect it to the Express application using Mongoose, define schemas and models for Project and Contact documents, and implement full Create, Read, Update, and Delete routes for both collections.

Algorithm

1. Create a free MongoDB Atlas cluster, configure database access credentials and network access, and obtain the connection string.
2. Install Mongoose in the Express project and connect to Atlas using the connection string inside the server file, with appropriate error handling.
3. Define a Project schema with fields such as title, description, image URL, and rating, and compile it into a model.
4. Define a Contact schema with fields such as name, email, message, and a creation timestamp, and compile it into a model.
5. Update the GET projects route to fetch all documents from MongoDB using the model's find method with async/await.
6. Add a POST projects route that creates a new project document from the request body.
7. Add a PUT route that updates an existing project document identified by its id, returning the updated document.
8. Add a DELETE route that removes a project document identified by its id.
9. Update the POST contact route to persist submitted messages as documents instead of storing them in an array.
10. Test all CRUD routes using a REST client, and verify the documents in the MongoDB Atlas collections view.

Viva Questions & Answers

Viva Voce Questions & Answers

Q1. What is MongoDB, and how does it differ from a relational database?

Ans: MongoDB is a NoSQL, document-oriented database that stores data as flexible, JSON-like documents in collections, whereas a relational database stores data in fixed-schema tables with rows and requires predefined relationships between tables.

Q2. What is Mongoose, and what advantage does it provide over the native MongoDB driver?

Ans: Mongoose is an Object Data Modeling library for MongoDB and Node.js that lets developers define schemas with data types and validation rules, providing structure and built-in validation on top of MongoDB's otherwise schema-less documents.

Q3. What do the four CRUD operations stand for?

Ans: Create, Read, Update, and Delete — the four basic operations used to manage persistent data.

Q4. Why is a 'return the updated document' option typically passed when updating a document by its id?

Ans: By default, an update-by-id operation returns the document as it was before the update; passing this option instructs Mongoose to return the updated document instead.

Q5. What is a Mongoose schema, and what role does it play given that MongoDB itself is schema-less?

Ans: A schema defines the shape, data types, defaults, and validation rules for documents within a collection at the application level, enforcing structure and consistency even though the underlying database does not require a fixed schema.

Q6. Why should database connection logic include error handling?

Ans: Because the connection is asynchronous and can fail, for example due to incorrect credentials or network access issues, unhandled connection errors can crash the server or leave it silently unable to serve data.

EXPERIMENT 11

Full-Stack Integration (React + Express + MongoDB)

Aim

To integrate the React frontend, Express API, and MongoDB database into a single, cohesive MERN application.

Description

This experiment brings together all previous components into one working full-stack application. Students connect the React portfolio to the live Express API endpoints, ensure the Contact form submits to and is persisted in MongoDB, handle loading and error states in the UI, and organise the project into a clean client and server structure suitable for deployment.

Algorithm

1. Organise the project into separate client and server folders, or a suitable monorepo structure.
2. In the React app, replace any static or mock project data with a data-fetching effect that calls the live projects endpoint on the Express server and stores the result in state.
3. Add loading and error state variables in the relevant components, displaying a loading indicator while the request is pending and an error message if it fails.
4. Update the ContactForm component to submit validated form data to the Express server's contact endpoint instead of only performing client-side validation.
5. Configure a development proxy or an environment variable for the API base URL to simplify API calls and avoid hardcoding the server address.
6. Verify that CORS is correctly configured on the Express server to accept requests from the React development server's origin.
7. Test the complete flow end-to-end: loading the portfolio fetches live projects from MongoDB via Express, and submitting the contact form creates a new document in the Contact collection.
8. Add basic UI feedback, such as a success message, confirming that a contact submission was received.
9. Review and refactor the code for consistency, removing unused mock data, and confirm the theme toggle and skills table still work correctly with live data.

Viva Questions & Answers

Viva Voce Questions & Answers

Q1. What does the acronym MERN stand for, and what role does each technology play?

Ans: MongoDB (database), Express.js (backend web framework), React (frontend UI library), and Node.js (JavaScript runtime) — together forming a full JavaScript-based stack from database to browser.

Q2. Why is it good practice to separate the client and server into distinct folders or processes during development?

Ans: It keeps frontend and backend concerns independent, allows each to be developed, tested, and scaled separately, and mirrors how they are typically deployed as separate services.

Q3. What problem does a development proxy or an API base URL environment variable solve?

Ans: It avoids hardcoding the backend's address in the frontend code and prevents cross-origin issues during local development by routing frontend API calls to the backend consistently.

Q4. Why is it important to handle loading and error states explicitly when fetching data from an API?

Ans: Network requests are asynchronous and can fail or take time; without explicit loading and error states, users may see a blank or broken UI with no indication of what is happening.

Q5. In a full-stack flow, where should form validation occur, and why?

Ans: On both the client, for immediate feedback and a better user experience, and the server, as the authoritative check, since client-side checks can be bypassed, ensuring both usability and data integrity.

Q6. What is the typical request path when a user submits the Contact form in the fully integrated application?

Ans: The React ContactForm sends a request to the Express API's contact endpoint; Express validates the data and passes it to the Contact model, which persists it as a document in MongoDB Atlas, after which Express responds so the UI can update.

Deployment of the MERN Portfolio Application

Aim

To deploy the complete MERN portfolio application to the cloud so that it is publicly accessible.

Description

This experiment covers taking the full-stack application from a local development environment to a live, publicly accessible deployment. Students prepare the React app for production, deploy the frontend to a static hosting service, deploy the Express and Node backend to a cloud platform, configure environment variables securely, and connect the deployed frontend to the deployed backend.

Algorithm

1. Prepare the Express server for production by moving sensitive values, such as the MongoDB connection string and port, into environment variables, and exclude them from version control.
2. Push the client and server code to a source control repository to enable deployment through the hosting platform's Git integration.
3. Deploy the backend to a cloud platform, configuring the build and start commands and setting the required environment variables in the platform's dashboard.
4. Update MongoDB Atlas's network access settings to allow connections from the deployed backend.
5. Build the React app for production, generating an optimised static bundle.
6. Deploy the built frontend to a static hosting service, pointing it at the production build output.
7. Set the frontend's API base URL to point to the deployed backend's live address instead of the local address.
8. Update the CORS configuration on the deployed Express server to allow requests from the deployed frontend's live domain.
9. Test the fully deployed application end-to-end from a browser, ideally on a different device or network, to confirm the portfolio loads live data and the contact form submits successfully.
10. Document the live frontend and backend URLs, along with any deployment issues encountered and how they were resolved.

Viva Questions & Answers

Viva Voce Questions & Answers

Q1. Why should sensitive values, such as database connection strings, never be hardcoded into source code?

Ans: Hardcoded secrets can be committed to version control and exposed publicly, especially in open repositories; environment variables keep such values out of the codebase and configurable per environment.

Q2. What is the purpose of building a React app before deployment?

Ans: The build process bundles, minifies, and optimises the application into static files, producing a smaller, faster-loading production version compared to the unoptimised development server.

Q3. Why do the frontend and backend often need to be deployed to different types of hosting platforms?

Ans: Once built, the frontend is a set of static files best served by a static host for speed and low cost, while the backend is a persistently running process that needs a platform capable of executing server code and maintaining a live database connection.

Q4. Why must CORS settings be updated again after deployment, even if they worked during local development?

Ans: The deployed frontend runs on a different domain than the local development server, so the backend's CORS configuration must explicitly allow that new production origin, or cross-origin requests will be blocked by the browser.

Q5. Why is it necessary to update MongoDB Atlas's network access settings after deploying the backend?

Ans: Atlas blocks connections from IP addresses that are not on its access list by default, and since the deployed backend runs from the cloud platform's servers rather than the developer's local machine, its access must be explicitly permitted.

Q6. What is the difference between a development environment and a production environment in the context of this deployment?

Ans: The development environment runs locally with features like hot-reloading and verbose debugging, while the production environment runs the optimised build on remote servers, uses environment-specific configuration, and should have minimal debug output with more robust error handling.